Full Stack Assignment – 26

Assignment ID: FSD-26

Title: Internal Asset Tracking Management System

Duration: 3 Hours

1️⃣ Objective

Design and develop an **Internal Asset Tracking Management System** that allows administrators to manage organizational assets and users to request and use assigned assets.
The system must enforce asset return rules and ensure **data persistence using a database**.

2️⃣ User Roles

USER

- Register and log in to the system
- View assets assigned to them
- Request an asset
- Mark an asset as returned

ADMIN

- Register and log in to the system
- Create and manage assets
- View all asset requests and assignments
- Assign and unassign assets to users

3️⃣ Functional Requirements

Authentication & Authorization

- Implement user registration and login
- Use **JWT-based authentication**
- Protect all backend APIs using JWT
- Enforce **role-based access control** on APIs and frontend views

Asset Management

- ADMIN can create assets with basic details (asset name, type, status)
- Asset data must be stored in the database
- Asset availability must be fetched dynamically

Asset Assignment

- USER can request an available asset

- ADMIN can assign assets to users
- Asset assignment and return status must persist in the database

4 Business Rule (Critical)

⚠️ **An asset must be returned before it can be reassigned to another user.**

Any attempt to reassign an asset that is not yet returned must be rejected.

5 Technical Constraints (Mandatory)

- All asset and assignment data **must persist in a database**
- Hardcoded or in-memory storage is **not allowed**
- Backend APIs must reflect actual database state
- JWT authentication and role validation are mandatory
- Use environment variables for sensitive configuration

6 Frontend Requirements

- Login and registration pages
- Role-based dashboards for USER and ADMIN
- Asset listing with current assignment status
- Asset request and return actions for USER
- Asset creation and assignment view for ADMIN

UI design is not evaluated; rule enforcement and correctness matter.

7 Submission Requirements (Strict)

GitHub Repository

- Backend source code
- Frontend source code
- Proper folder structure
- Clear and meaningful commit history

Live Deployment

- Live frontend deployment link
- Live backend deployment link
- Backend must be connected to the database

README.md (Mandatory)

The README file must include:

- Project overview

- Tech stack used
- User roles and permissions
- API endpoints
- Database schema or collections
- Steps to run the project locally
- Live frontend and backend links

8️⃣ Evaluation Note

Evaluation focuses on **state enforcement, role control, and end-to-end execution**.